Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Диагностика замедлений и ошибок

Симптом, метрика, причина, действие · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 Диагностика замедлений и ошибок 3

2 Инструменты 4

3 Жалоба пользователя с кодом запроса 5

4 Симптом → метрика → причина → действие 6

5 Команды 8

6 Профиль процессора и снимок памяти 9

7 Что передать разработчику 10

1 Диагностика замедлений и ошибок

Регламент для администратора стенда и разработчика (E51-05): как от жалобы пользователя или оповещения мониторинга дойти до причины. Порядок всегда один: **симптом → метрика → причина → действие**. Действие выбирается только после того, как причина подтверждена метрикой или журналом; «перезапустить и посмотреть» — не диагностика.

Команды даны для стенда из `docker-compose.yml`; на стенде без контейнеров те же команды выполняются без `docker compose exec`.

2 Инструменты

ИНСТРУМЕНТ	ЧТО ДАЁТ	КАК ВКЛЮЧИТЬ	ГДЕ СМОТРЕТЬ
Код запроса `X-Request-Id`	Связывает жалобу, строку nginx, журнал API и медленные запросы к БД	Всегда; nginx передаёт свой код или создаёт новый	Заголовок ответа; поле `requestId` в ошибке; «Код запроса: 1a2b3c4d» в сообщении интерфейса при ошибке 5xx
Журнал nginx	Каждый запрос с кодом `rid=` и временем ответа	Всегда	`docker compose logs web`
Журнал API	Медленные запросы и ответы 5xx: маршрут, код, длительность, сотрудник, адрес	`REQUEST_LOG=slow` (по умолчанию), `SLOW_REQUEST_MS`	`docker compose logs server` — строки JSON при `LOG_FORMAT=json`
Медленные запросы к БД из API	Модель и операция Prisma или текст SQL — без параметров; код запроса API	`SLOW_QUERY_MS=100` и перезапуск `server`	Тот же журнал, `"msg":"slow query"`
Журнал медленных запросов PostgreSQL	Полный текст запроса дольше порога	`PG_SLOW_MS` (по умолчанию 1000 мс)	`docker compose logs postgres`
`pg_stat_statements`	Накопленная статистика запросов: суммарное и среднее время, вызовы, чтение с диска	Загружается при старте `postgres` (compose 1.0.0)	`deploy/postgres/top-queries.sql`
Метрики Prometheus	Задержки по маршрутам, ядра и память процессов, задержка цикла событий, соединения с БД, очередь писем	Профиль `monitoring` или Prometheus организации	Панель Grafana `deploy/monitoring/grafana-dashboard.json`
Профиль процессора	Где процесс API тратит время	`NODE_OPTIONS=--cpu-prof`	Файлы `.cpuprofile` — Chrome DevTools, вкладка Performance
Снимок памяти	Что занимает кучу процесса API	`NODE_OPTIONS=--heapsnapshot-signal=SIGUSR2`	Файлы `.heapsnapshot` — Chrome DevTools, вкладка Memory

Журнал API не содержит паролей, токенов, тел запросов и параметров SQL: в нём только маршрут без строки запроса, код ответа, длительность, идентификатор сотрудника и адрес клиента. Его можно передавать разработчику без обезличивания.

3 Жалоба пользователя с кодом запроса

При ошибке сервера интерфейс показывает «Код запроса: 1a2b3c4d» — первые 8 символов кода. По нему находятся все записи этого запроса:

```
docker compose logs --no-log-prefix server | grep '"reqId":"1a2b3c4d'
docker compose logs --no-log-prefix web | grep 'rid=1a2b3c4d'
```

В журнале API у ответа 5xx есть строка `"msg":"unhandled error"` со стеком — её и передают разработчику. Если пользователь жалуется на медленную работу без ошибки, код в интерфейсе не показывается: ищите по времени, логину (идентификатор сотрудника — поле `user`) и маршруту.

4 Симптом → метрика → причина → действие

СИМПТОМ	МЕТРИКА ИЛИ ЖУРНАЛ	ПРИЧИНА	ДЕЙСТВИЕ
Медленно у всех, оповещение `VacationReadLatencyHigh` или `VacationWriteLatencyHigh`	`rate(process_cpu_seconds_total[5m])` близко к числу процессов API; `vp_eventloop_lag_p99_seconds` > 0,2 с	Процессам API не хватает ядер	Добавьте ядра или второй экземпляр (руководство по развёртыванию, «Производительность»); если ядер достаточно — профиль процессора: какая операция занимает ядро
Медленно у всех, ядра API свободны	`vp_db_connections{state="active"}` растёт; в `top-queries.sql` один запрос — больше трети суммарного времени	Дорогой запрос к БД или полный просмотр большой таблицы	`EXPLAIN (ANALYZE, BUFFERS)` запроса; отсутствующий индекс или неудачный план — разработчику с выдержкой из `top-queries.sql`
Медленно у всех, ядра API и БД свободны	`vp_db_connections{state="idle in transaction"}` ; ожидания блокировок в `pg_stat_activity`	Долгая транзакция держит блокировку	Запрос «Кто кого ждёт» ниже; завершите зависший сеанс `pg_terminate_backend` ; найдите его источник по `application_name` и времени
Медленный один экран	Журнал API: `"msg":"slow request"` с этим маршрутом; при `SLOW_QUERY_MS` — запросы к БД с тем же `reqId`	Дорогая операция или запрос к БД	Сгруппируйте медленные запросы по маршруту (команда ниже); если время уходит в БД — в `top-queries.sql` , иначе — профиль процессора
«Сервис перегружен» (`503`)	`vp_db_errors_total{code="P2024"}` — нет свободного соединения в пуле	Пул исчерпан долгими запросами или мал для нагрузки	Сначала долгие запросы (`pg_stat_activity`); если их нет — `DB_POOL_SIZE` в пределах `max_connections`
Ошибки 5xx, оповещение `VacationServerErrors`	Журнал API: `"msg":"unhandled error"` со стеком	Ошибка в коде или в данных	Стек и код запроса — разработчику; воспроизведение — по маршруту и сотруднику

СИМПТОМ	МЕТРИКА ИЛИ ЖУРНАЛ	ПРИЧИНА	ДЕЙСТВИЕ
Вход дольше секунды утром или после рассылки	`slow request` на `/api/v1/auth/login`; ядра API заняты	Проверка пароля намеренно дорогая (scrypt): волна входов занимает ядра	Ожидаемо при массовом входе; если p95 входа больше 1 с при обычной нагрузке — ядра API (таблица сайзинга)
Память процесса API растёт часами	`process_resident_memory_bytes` растёт без возврата после спада нагрузки	Утечка памяти	Два снимка памяти с интервалом в час — разработчику; временно — перезапуск `server` в нерабочее время
Уведомления и график не обновляются сами	`vp_sse_connections` около нуля при работающих пользователях	Прокси организации буферизует или обрывает поток событий	Руководство по развёртыванию, «Поток событий через прокси»
Письма не уходят	`vp_mail_outbox_oldest_seconds`, оповещение `VacationMailStuck`	Почтовый релей недоступен или отвергает	Руководство по развёртыванию, «Почта»
База растёт быстрее ожидаемого	Размеры таблиц (запрос ниже)	Нет очистки или раздувание после массовых изменений	Сроки хранения — руководство по развёртыванию, «Обслуживание»; `VACUUM` больших таблиц

5 Команды

Медленные маршруты за последний час и сколько раз каждый:

```
docker compose logs --no-log-prefix --since 1h server \
  | jq -rR 'fromjson? | select(.msg=="slow request") | .route' | sort | uniq -c | sort -rn | head
```

Запросы к БД одного медленного запроса API (при `SLOW_QUERY_MS`):

```
docker compose logs --no-log-prefix server | jq -cR 'fromjson? | select(.reqId=="<код>")'
```

Самые затратные запросы БД с момента сброса статистики:

```
docker compose exec -T postgres psql -U vacation -d vacation -f - < deploy/postgres/top-queries.sql
docker compose exec postgres psql -U vacation -d vacation -c 'SELECT pg_stat_statements_reset()'   #
начать замер заново
```

Активные запросы и кто кого ждёт:

```
SELECT pid, now() - query_start AS running, state, wait_event_type, wait_event,
       pg_blocking_pids(pid) AS blocked_by, left(query, 120) AS query
  FROM pg_stat_activity
 WHERE datname = current_database() AND state <> 'idle'
 ORDER BY running DESC;
```

Размеры таблиц:

```
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS size, n_live_tup, n_dead_tup,
last_autovacuum
  FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 15;
```

6 Профиль процессора и снимок памяти

Профиль и снимки включаются на время разбора и выключаются после: профиль пишется при штатном останове процесса, снимок памяти на 1–2 секунды останавливает процесс и занимает 30–100 МБ.

1. Создайте рядом с `docker-compose.yml` файл `docker-compose.override.yml`:

```
services:
  server:
    environment:
      NODE_OPTIONS: --cpu-prof --cpu-prof-dir=/tmp/prof --diagnostic-dir=/tmp/prof --heapsnapshot-
signal=SIGUSR2
```

2. `docker compose up -d server` — процессы API запускаются с профилированием.
3. Воспроизведите замедление: те же действия пользователей или нагрузочный прогон (`docs/performance/runbook.md`).
4. Снимок памяти одного процесса API — первый процесс в списке главный, снимок делается с рабочего:

```
docker compose exec server sh -c 'kill -USR2 $(pgrep -f "dist/main.js" | sed -n 2p)'
```

5. `docker compose stop server` — каждый процесс при штатном останове записывает профиль.
6. `docker compose cp server:/tmp/prof ./prof`, удалите `docker-compose.override.yml`, `docker compose up -d server`.

Профиль открывается в Chrome DevTools (вкладка Performance → «Load profile»): в режиме Bottom-Up сверху — функции, на которые ушло больше всего времени. Для утечки памяти нужны два снимка одного процесса с интервалом: в DevTools (вкладка Memory) второй снимок сравнивается с первым в режиме Comparison.

7 Что передать разработчику

ЧТО	ОТКУДА
Время, маршрут, код запроса, сотрудник	Жалоба, журнал API
Строки журнала API этого запроса, включая стек	`grep` по коду запроса
Выдержка из `top-queries.sql`	Если время уходит в БД
Графики p95 по маршрутам, ядер и памяти процессов, соединений с БД за час до и после	Grafana
Профиль процессора или два снимка памяти	Если причина в процессе API
Версия, число процессов, ядра и память узла, численность сотрудников	`docker compose exec server node -p "require('./package.json').version"`, `.env`, `nproc`, `free -g`